Blockchain: Smart Contracts
Lecture 1

Please join course TEAMS: sxthogt

Grading

- Laboratory activity :--**50%**
- Final exam: **-- 50%**
  - Final Written Exam (open books)
- **To pass the exam you have to obtain minim 5 at the final exam and the final grade is minimum 5**

Course Content

- In this course we will discuss multiple blockchain related topics.
- At the laboratory you are going to complete some small assignments.

NOTE

- **Our slides are based on different blockchain courses and tutorials freely available online.**

What is a blockchain for?

Abstract answer: a blockchain provides
coordination between many parties,
when there is no single trusted party

if trusted party exists ⇒ no need for a blockchain

[financial systems: often no trusted party]

Blockchains: what is the new idea?

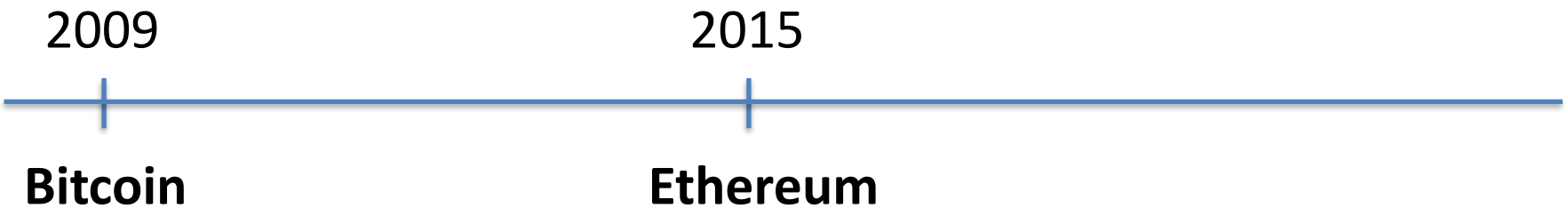
2009

Bitcoin

Several innovations:

- A practical **public append-only data structure**, secured by replication and incentives
- A fixed supply asset (BTC). Digital payments, and more.

Blockchains: what is the new idea?



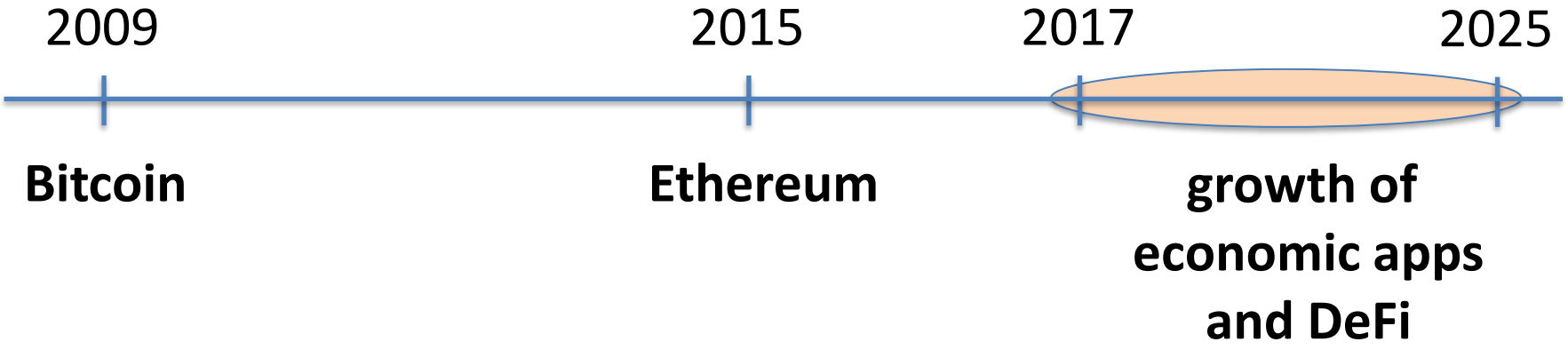
Several innovations:

- **Blockchain computer**: a fully programmable environment

  $\Longrightarrow$ public programs that manage digital and financial assets

- **Composability**: applications running on chain can call each other

Blockchains: what is the new idea?



Permissioned EVM blockchains by Stripe (Tempo) and Circle (Arc)

So what is this good for?

(1) Basic application: a digital currency (stored value)

- Current largest: Bitcoin (2009), Ethereum (2015), Solana (2020)
- Global: accessible to anyone with an Internet connection

Opinion

The New York Times

Bitcoin Has Saved My Family

"Borderless money" is more than a buzzword when you live in a collapsing economy and a collapsing dictatorship.

By Carlos Hernández

Mr. Hernández is a Venezuelan economist.

Feb. 23, 2019

What else is it good for?

(2) Decentralized applications (DAPPs)

- **DeFi**: financial instruments managed by public verifiable programs
  - examples: stablecoins, lending, exchanges, ….
- **Decentralized organizations** (DAOs): (decentralized governance)
  - DAOs for investment, for donations, for collecting art, etc.
- **x402**: AI payments -- agents and crawlers paying for services

---

(3) New programming model: writing decentralized programs

Assets managed by DAPPs

Aave v3	$41.8B	lending	multiple chains
LIDO	$39.5B	staking	Ethereum
WBTC (663 lines of Solidity code)	$14.7B	bridging	Ethereum
Morpho	$7.1B	lending	multiple chains
Uniswap v3	$3B	exchange	multiple chains

Sep. 2025

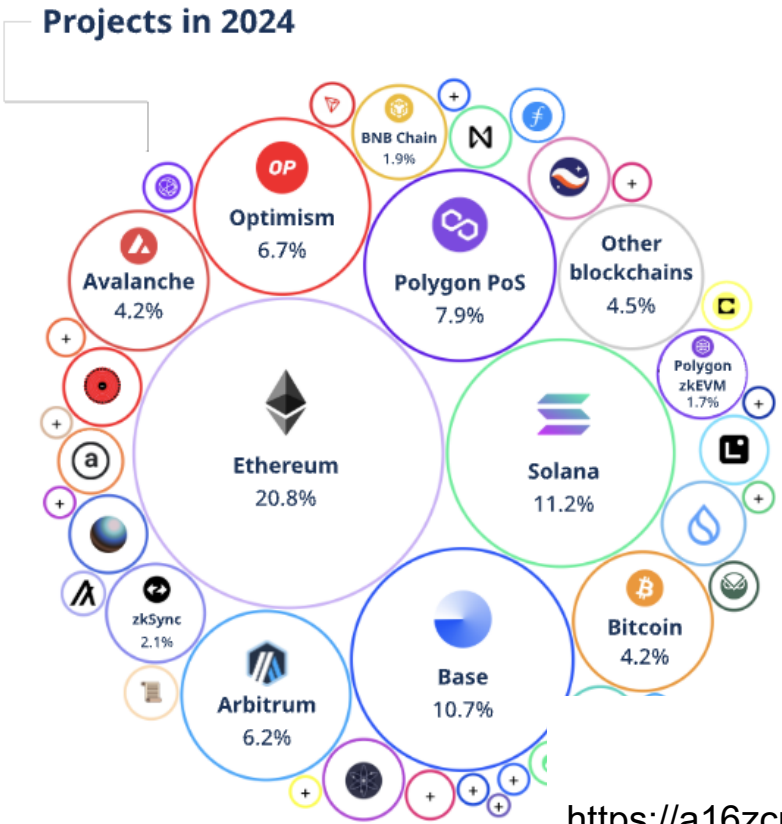
Transaction volumes

	24h volume	Sep. 2025
Bitcoin • BTC	$13.5B	
Ethereum • ETH	$2.35B	
Solana	$8.95B	

Cost to send USD internationally: $44 via wire transfer

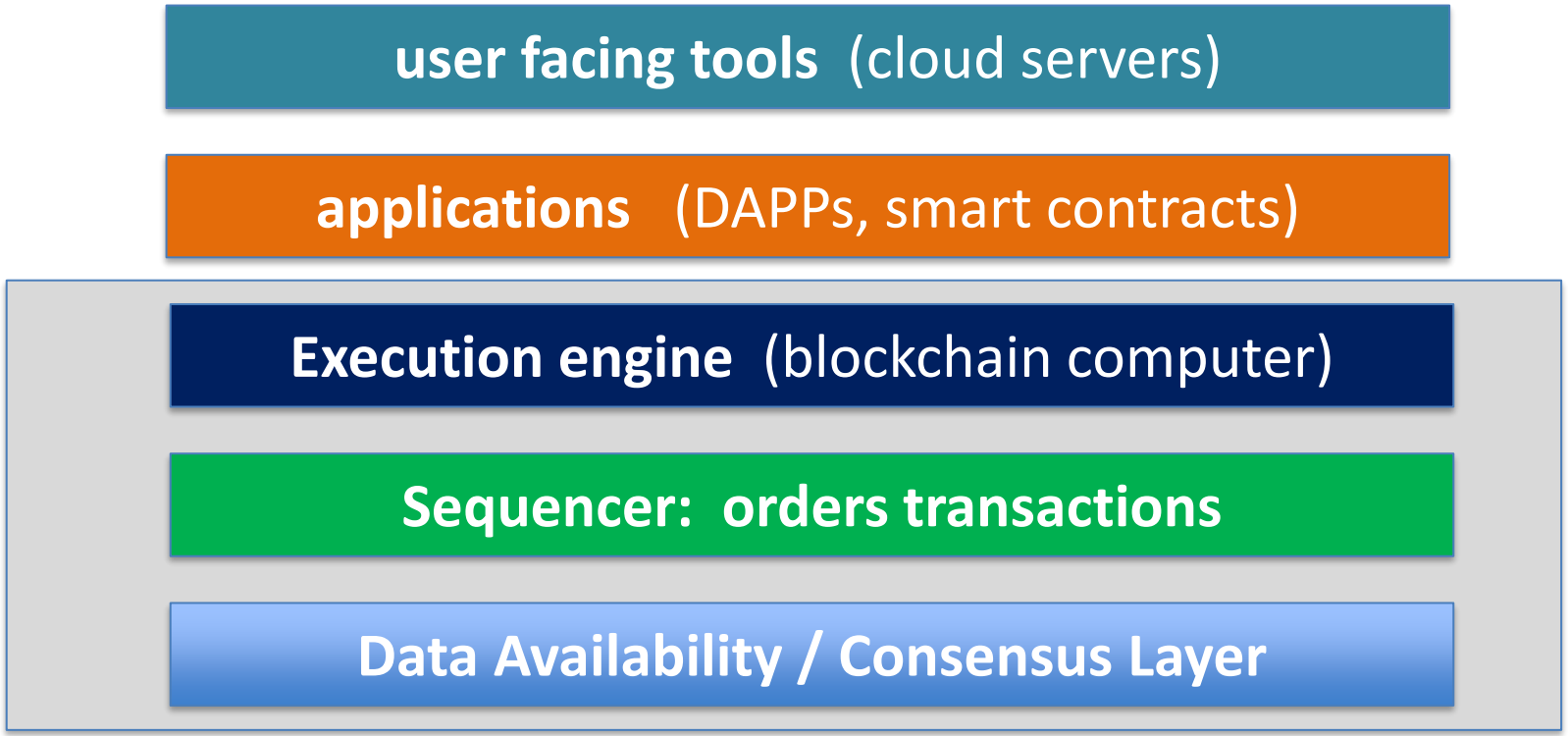
<$0.01 via USDC on Base

What chain are builders building on?



source: a16z state of crypto report 2024.
https://a16zcrypto.com/posts/article/state-of-crypto-report-2024/

What is a blockchain?



Consensus layer (informal)

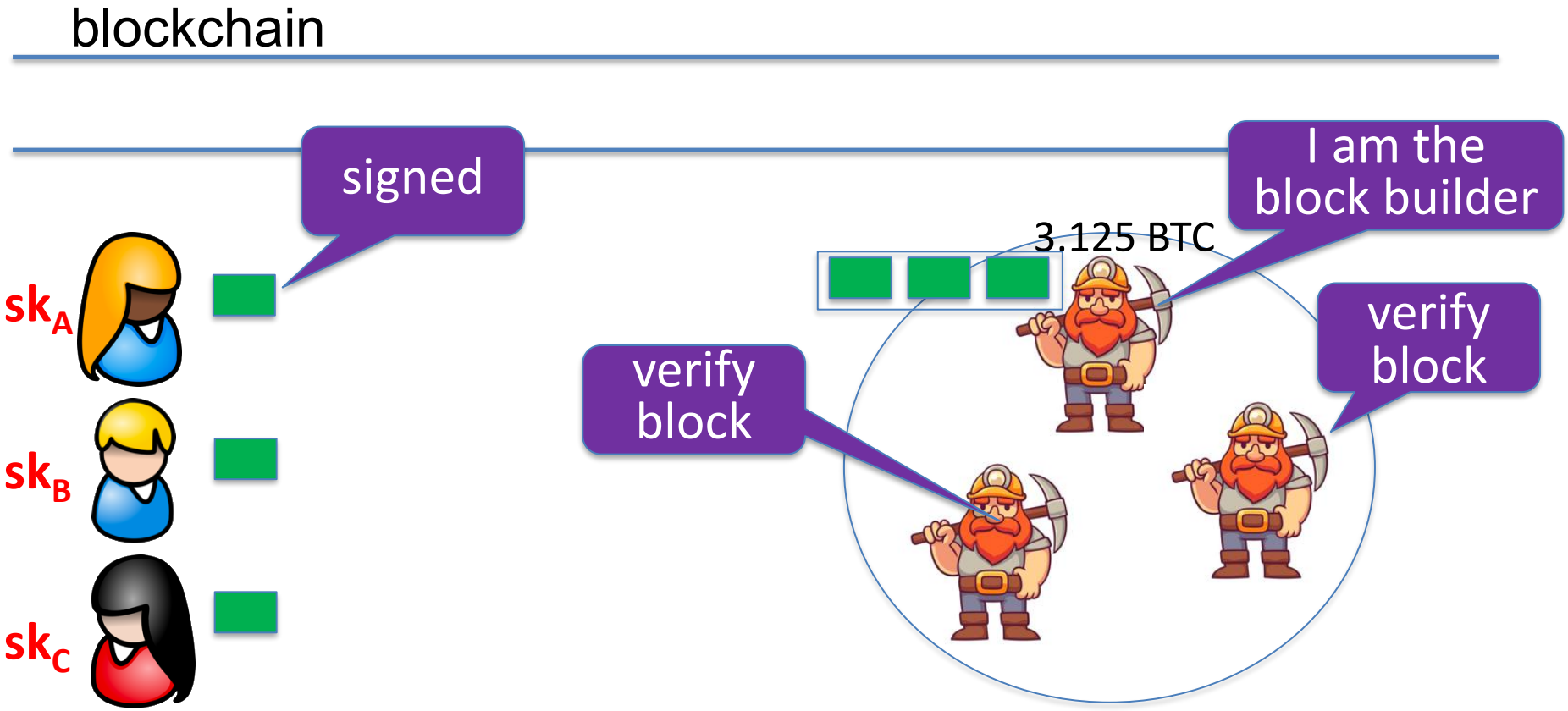
A **public** append-only data structure:

achieved by replication

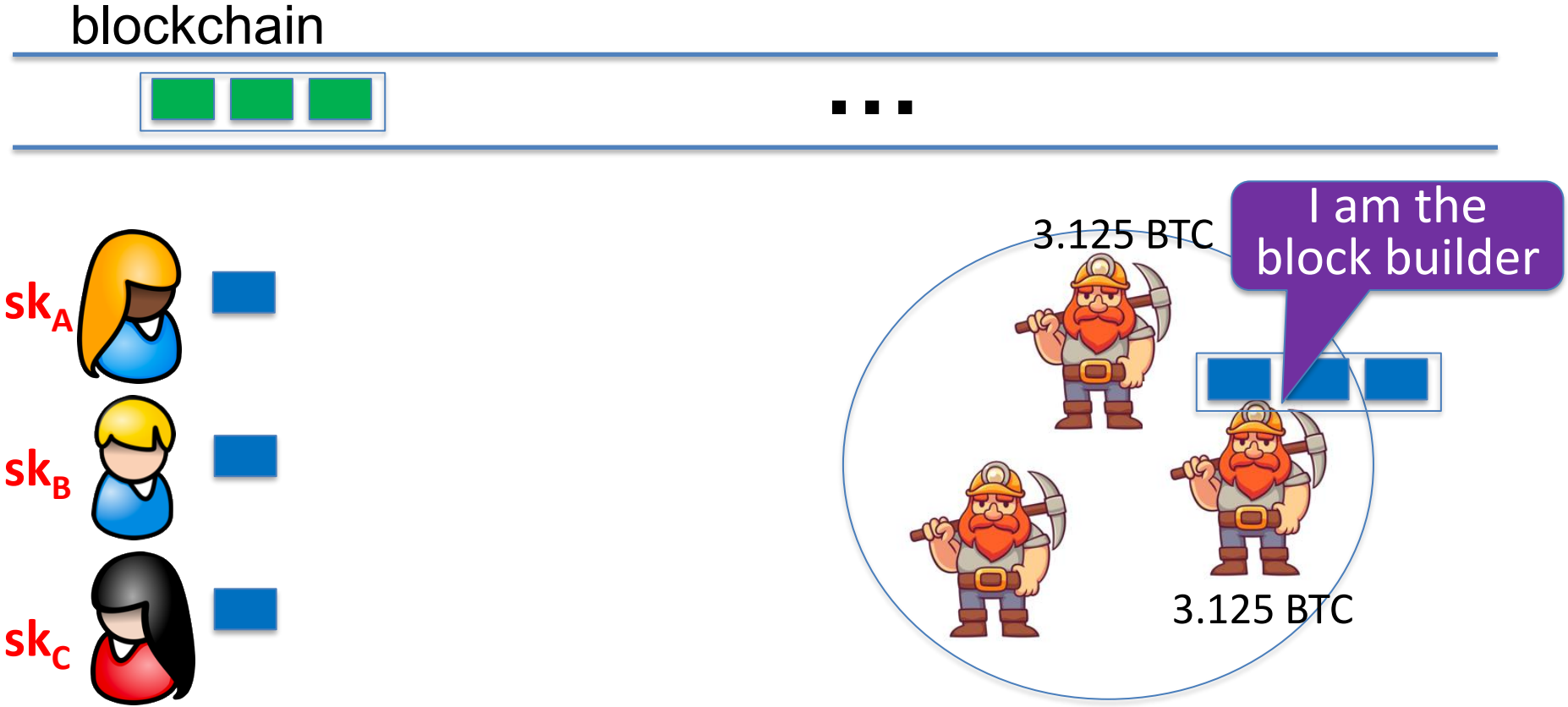
- **Persistence**: once added, data can never be removed*
- **Safety**: all honest participants have the same data**
- **Liveness:** honest participants can add new transactions
- **Permissionless(?)**: anyone can add data (no authentication)

Data Availability / Consensus layer

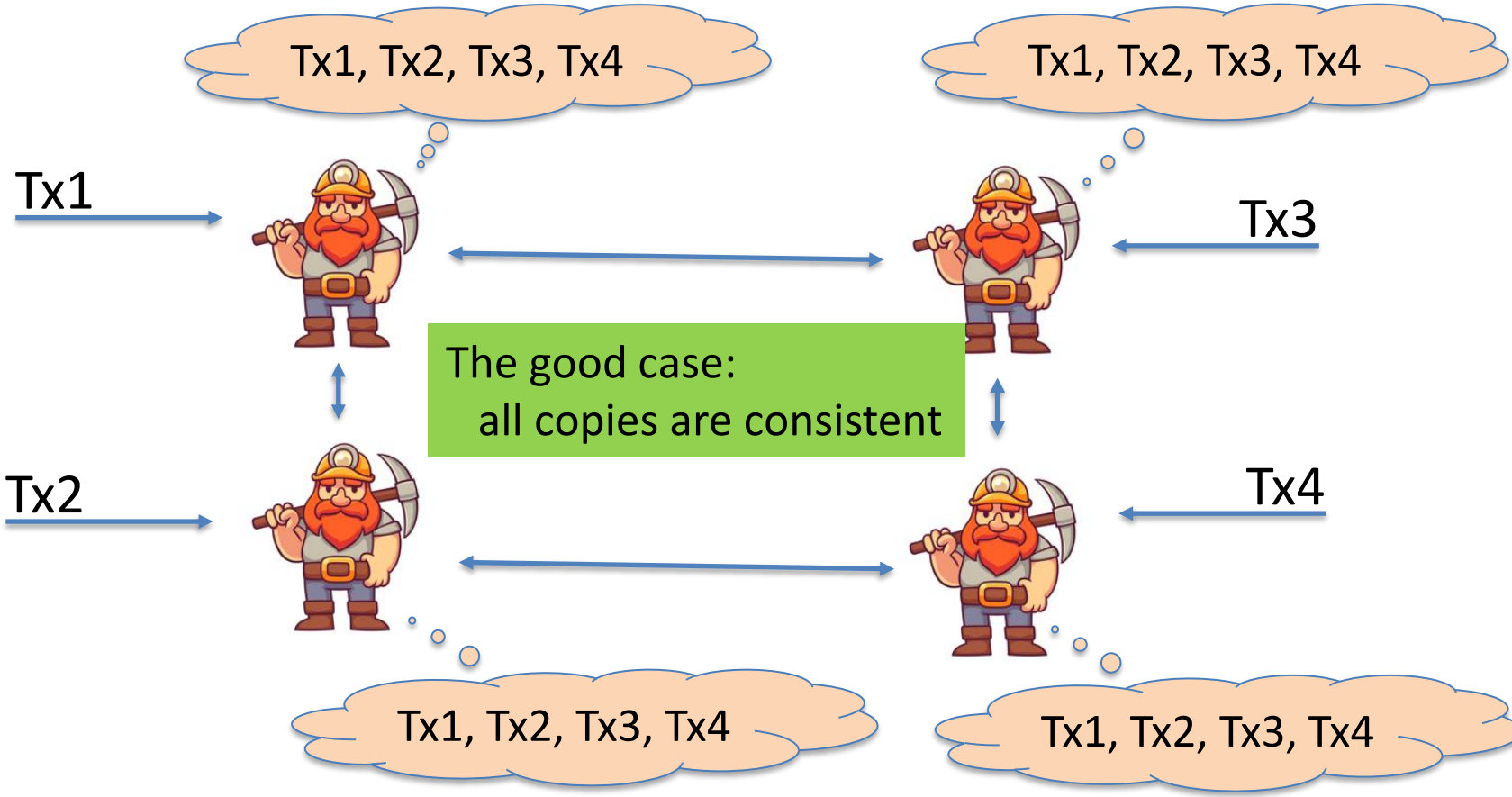
How are blocks added to chain?



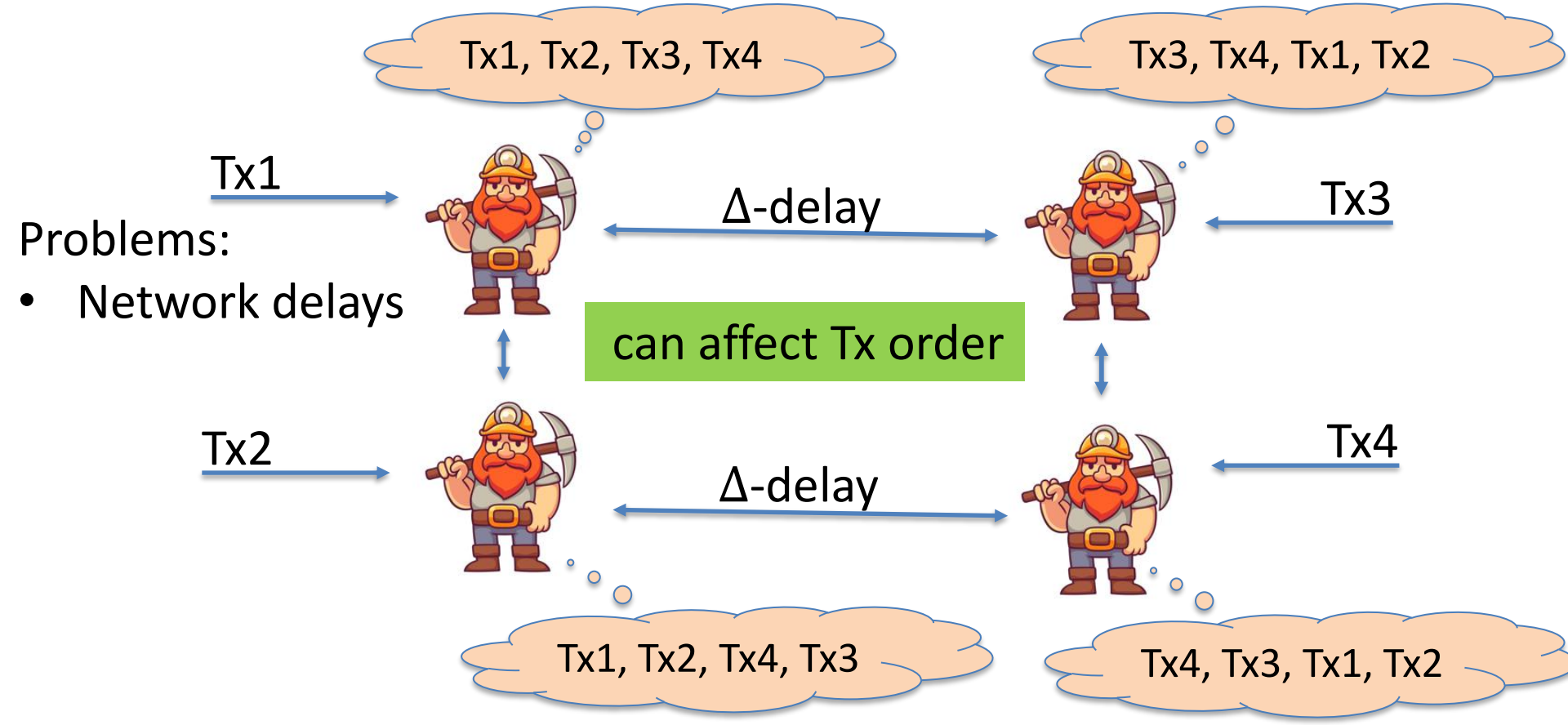
How are blocks added to chain?



Why is consensus a hard problem?



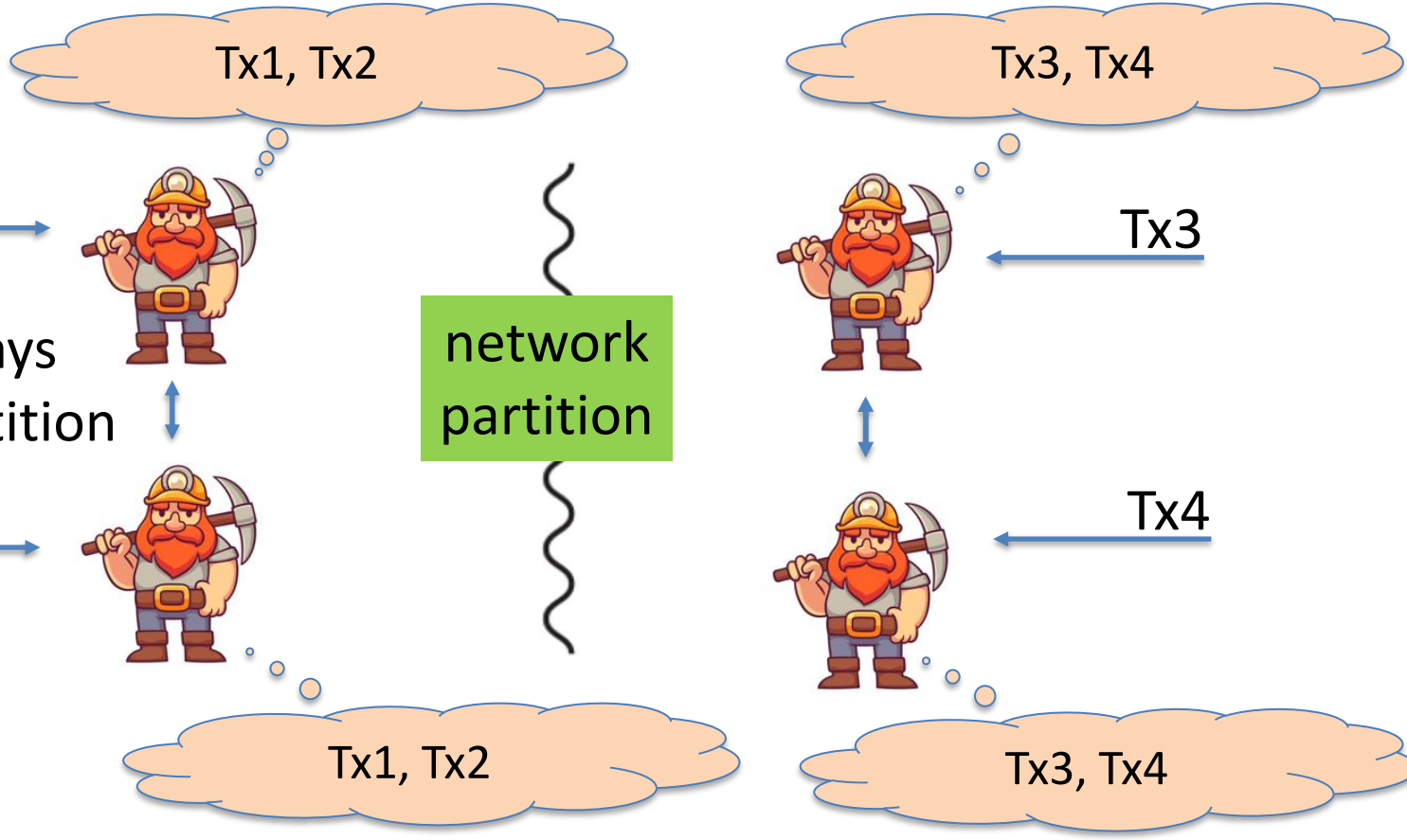
Why is consensus a hard problem?



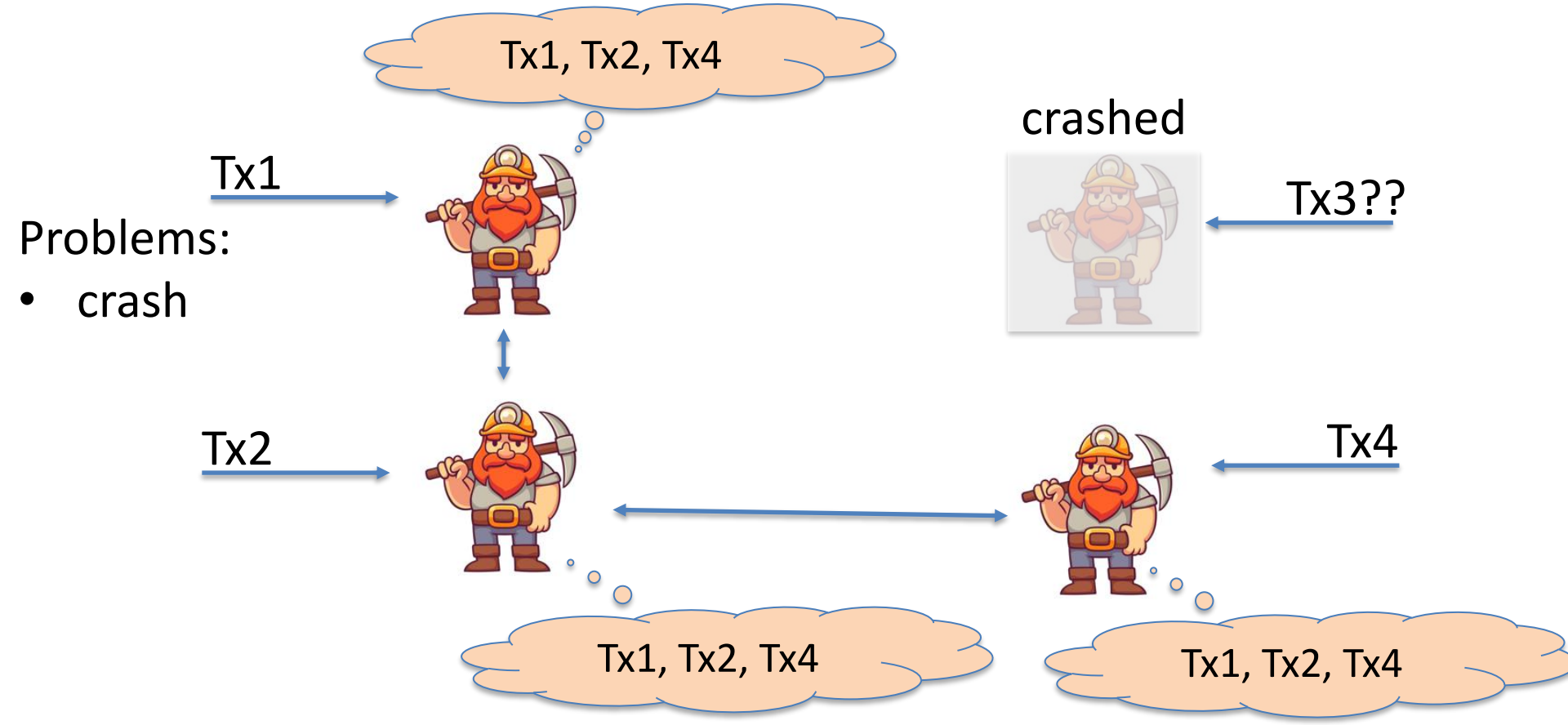
Why is consensus a hard problem?

Problems:

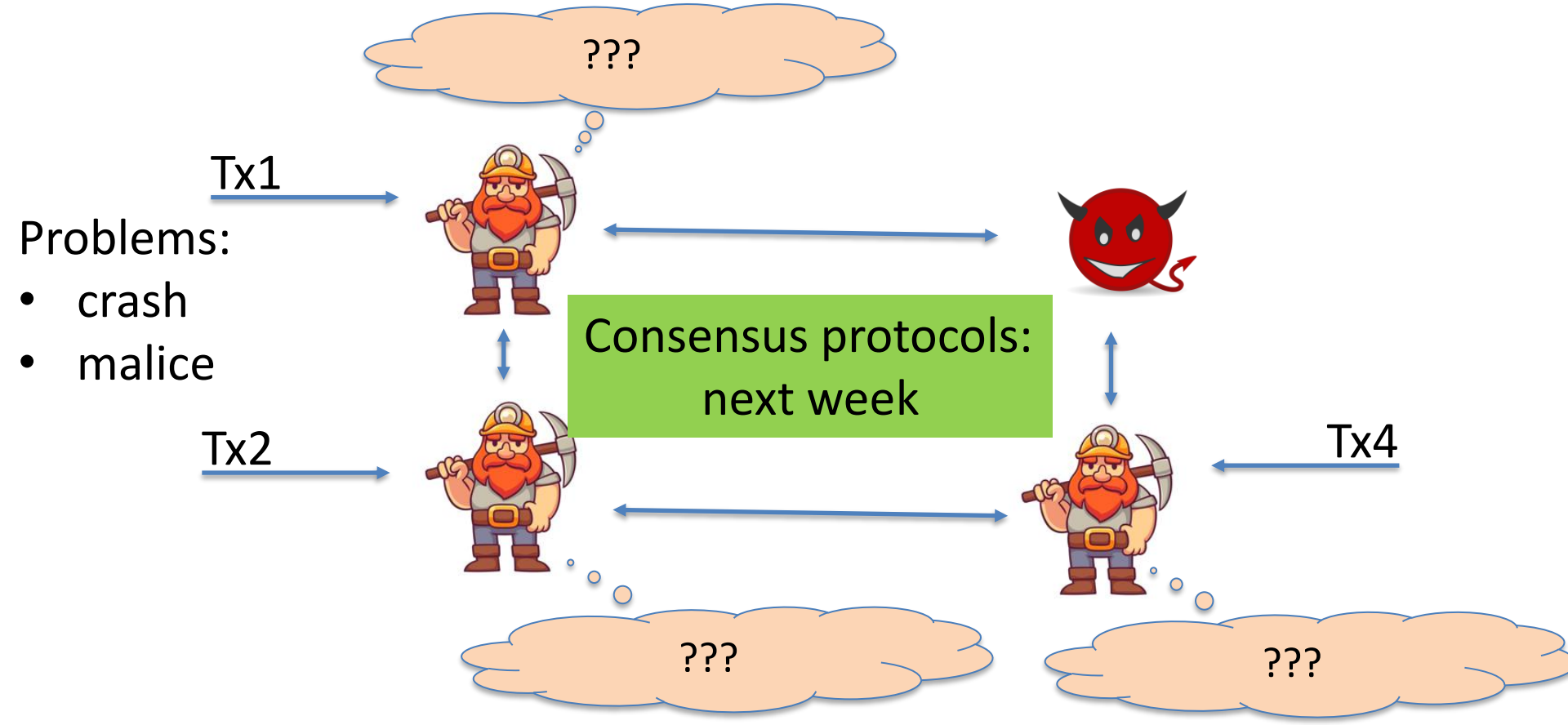
- Network delays
- Network partition



Why is consensus a hard problem?



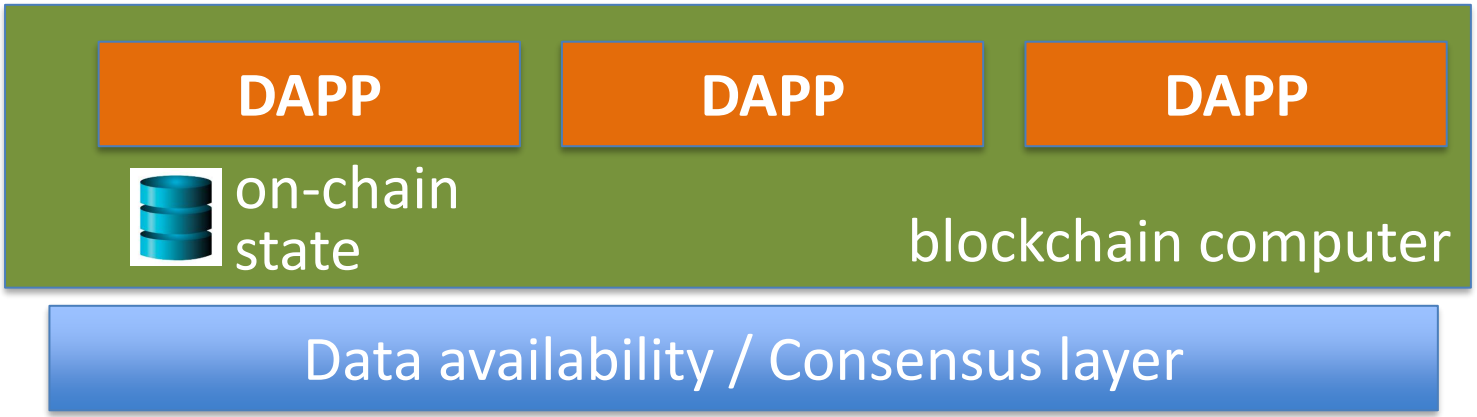
Why is consensus a hard problem?



Next layer: the blockchain computer

Decentralized applications (DAPPs):

- Run on blockchain: code and state are written on chain
- Accept Tx from users ⇒ state transitions are recorded on chain



Next layer: the blockchain computer

Top layer: user facing servers

end user

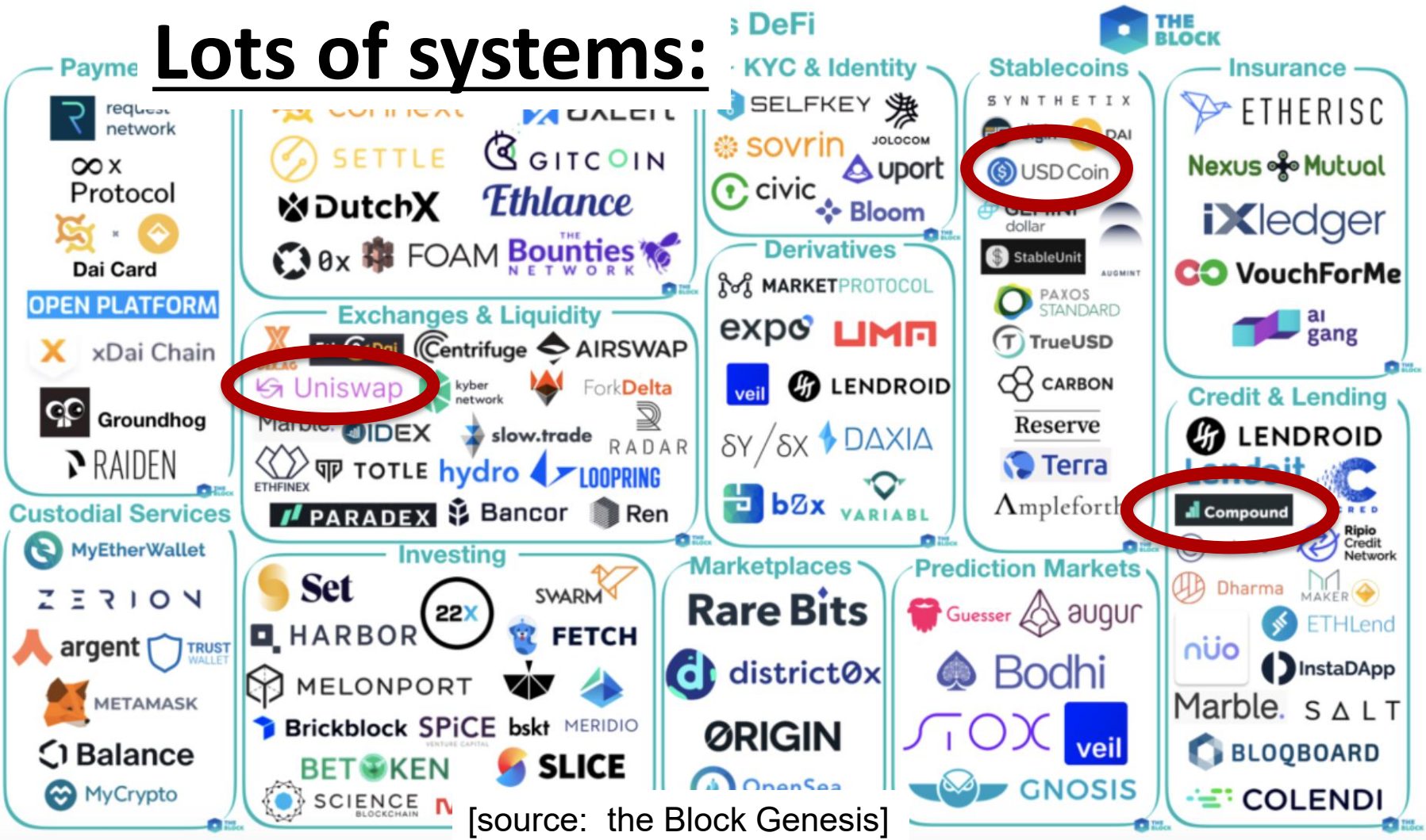
DAPP DAPP DAPP

on-chain state

blockchain computer

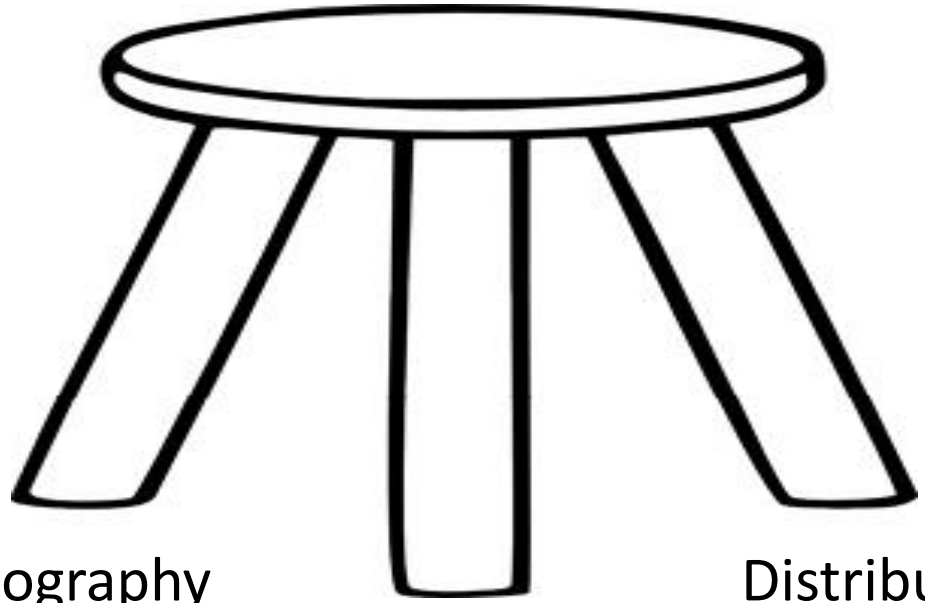
Data availability / Consensus layer

Lots of systems:



[source: the Block Genesis]

This course



Cryptography

Distributed systems

Economics

Course organization

1. The starting point: Bitcoin mechanics
2. Consensus protocols
3. Ethereum and decentralized applications
4. DeFi: decentralized applications in finance
5. Private transactions on a public blockchain
   (SNARKs and zero knowledge proofs)
6. Scaling the blockchain: getting to 1M Tx/sec
7. Interoperability among chains: bridges and wrapped coins

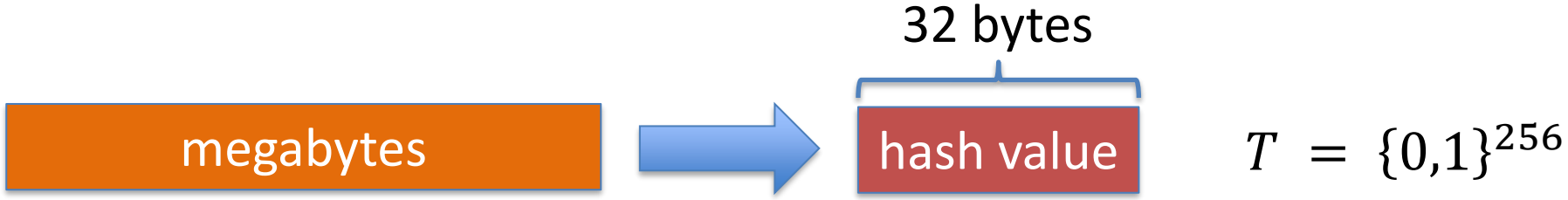
Let’s get started …

Cryptography Background

(1) cryptographic hash functions

An efficiently computable function $H: M \rightarrow T$

where $|M| \gg |T|$



Collision resistance

Def: a **collision** for $H: M \to T$ is pair $x \neq y \in M$ s.t. $H(x) = H(y)$

$|M| \gg |T|$ implies that many collisions exist

Def: a function $H: M \to T$ is **collision resistant** if it is "hard" to find even a single collision for H (we say H is a CRF)

Example: **SHA256**: $\{x : \text{len}(x) < 2^{64} \text{ bytes}\} \to \{0,1\}^{256}$

(output is 32 bytes)

Application: committing to data on a blockchain

Alice has a large file m. She posts $h = H(m)$ (32 bytes)

Bob reads h. Later he learns m' s.t. $H(m') = h$

H is a CRF $\Rightarrow$ Bob is convinced that $m' = m$

(otherwise, m and m' are a collision for H)

We say that $h = H(m)$ is a **binding commitment** to m

(note: not hiding, h may leak information about m)

Committing to a list (of transactions)

Alice has $S = (m_1, m_2, \ldots, m_n)$

32 bytes

Goal:

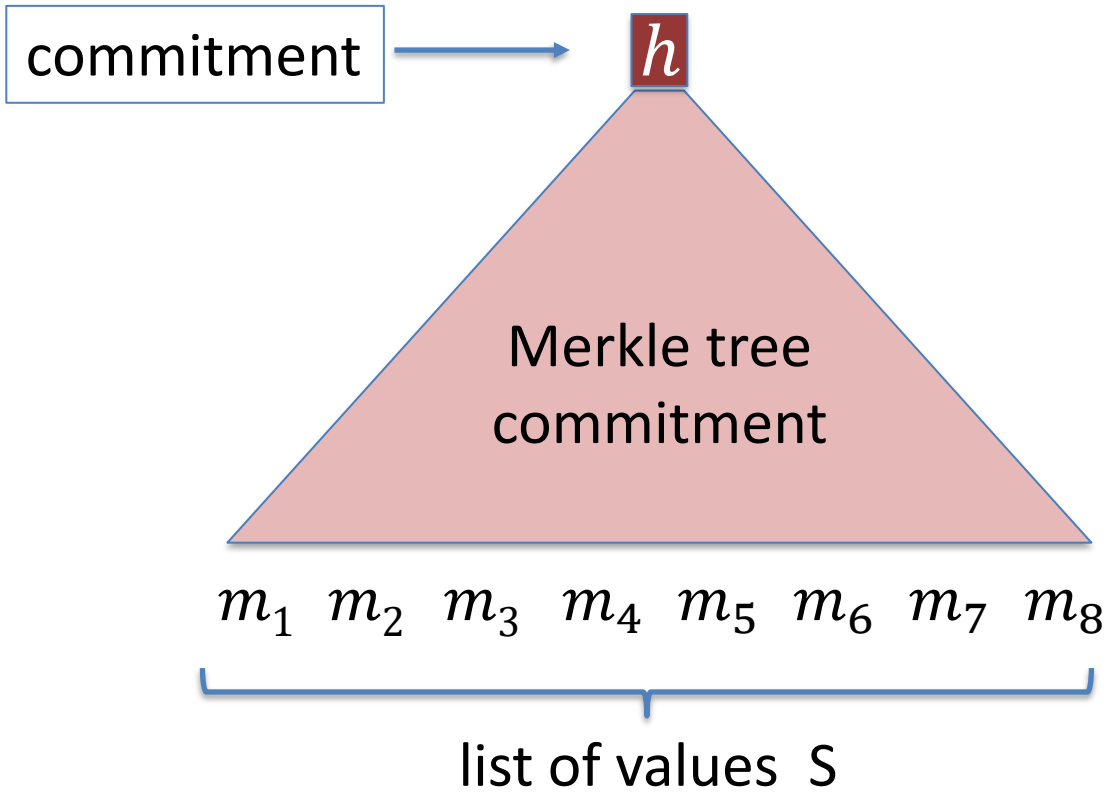
- Alice posts a short binding commitment to S, $h = \text{commit}(S)$
- Bob reads h. Given $(m_i,\ \text{proof}\ \pi_i)$ can check that $S[i] = m_i$

Bob runs $\text{verify}(h, i, m_i, \pi_i) \rightarrow$ accept/reject

security: adv. cannot find (S, i, m, π) s.t. $m \neq S[i]$ and

$\text{verify}(h, i, m, \pi) = \text{accept}$ where $h = \text{commit}(S)$

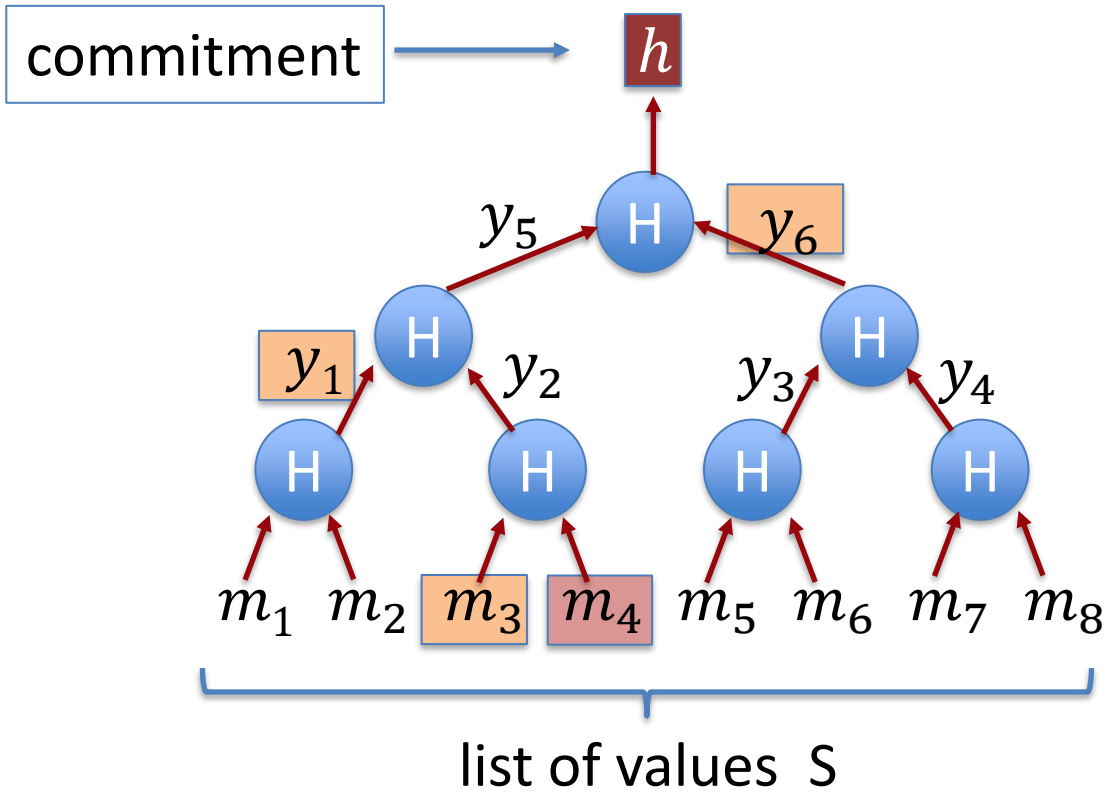
Merkle tree (Merkle 1989)



Goal:

- commit to list S of size n
- Later prove $S[i] = m_i$

Merkle tree (Merkle 1989) [simplified]



Goal:

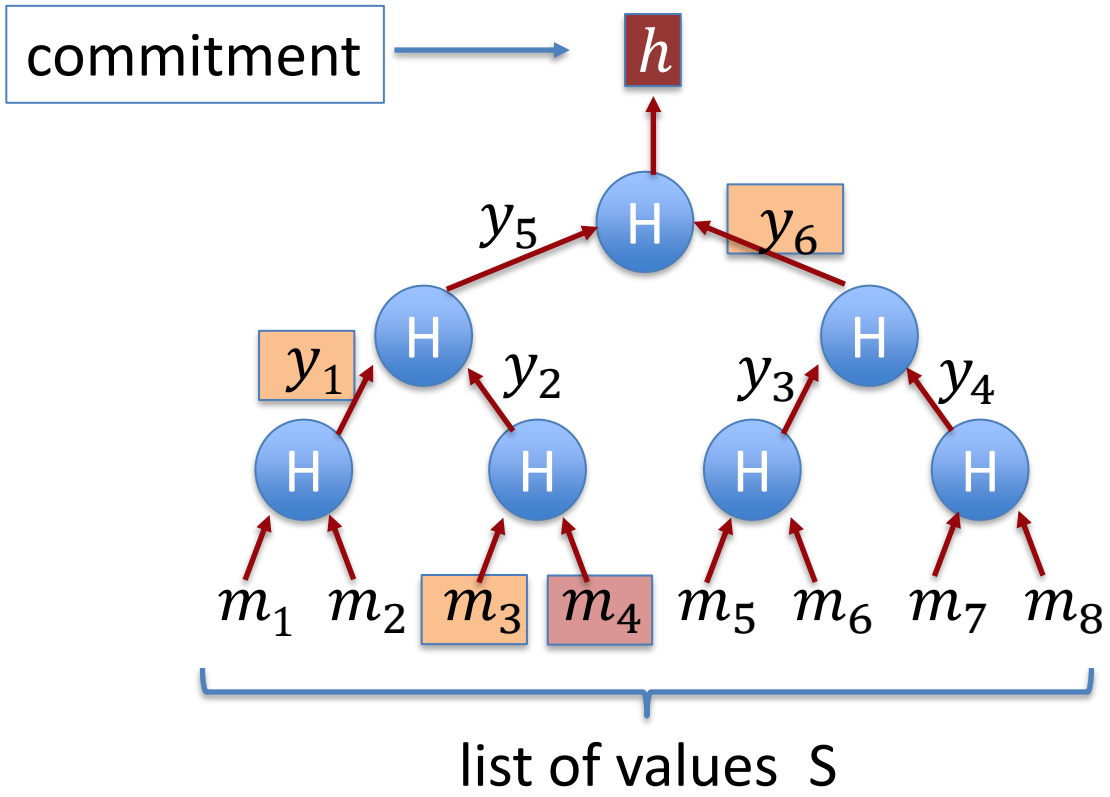
- commit to list S of size n
- Later prove $S[i] = m_i$

To prove $S[4] = m_4$,

proof $\pi = (m_3, y_1, y_6)$

length of proof: $\log_2 n$

Merkle tree (Merkle 1989) [simplified]



To prove $S[4] = m_4$,

proof $\pi = (m_3, y_1, y_6)$

Bob does:

$y_2 \leftarrow H(m_3, m_4)$

$y_5 \leftarrow H(y_1, y_2)$

$h' \leftarrow H(y_5, y_6)$

accept if $h = h'$

Merkle tree (Merkle 1989)

Thm: For a given n: if H is a CRF then

adv. cannot find (S, i, m, π) s.t. $|S| = n$, $m \neq S[i]$,

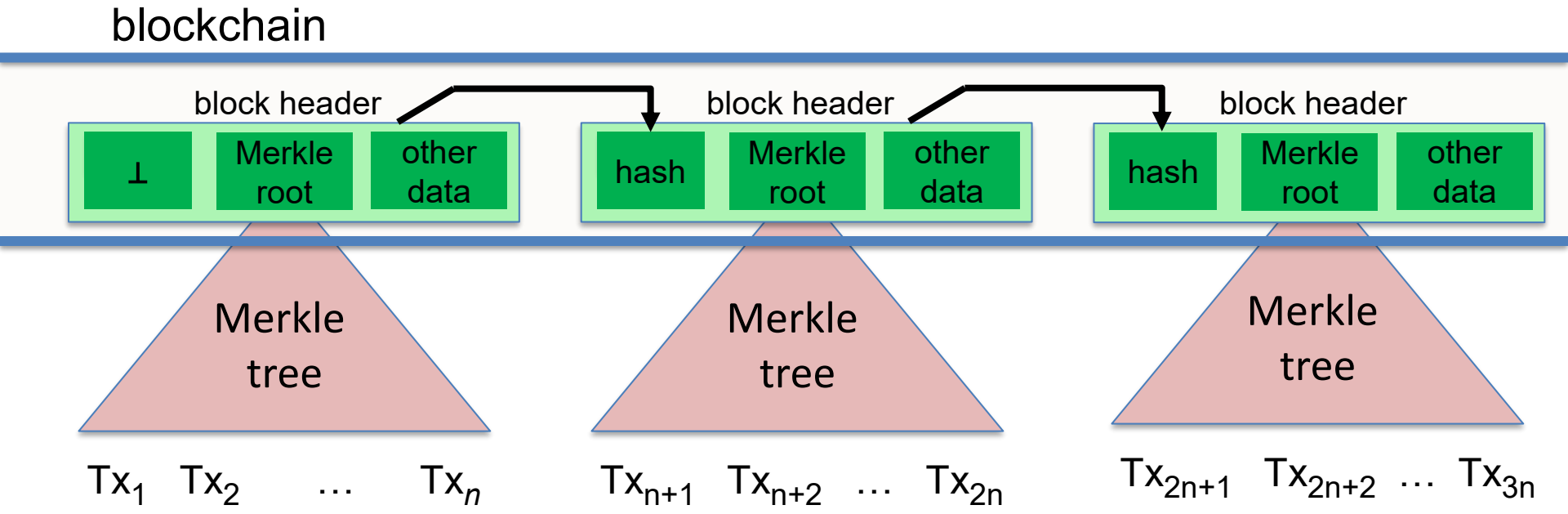
$$h = \text{commit}(S), \text{ and } \text{verify}(h, i, m, \pi) = \text{accept}$$

(to prove, prove the contra-positive)

How is this useful? To post a block of transactions S on chain suffices to only write commit(S) to chain. Keeps chain small.

$\Rightarrow$ Later, can prove contents of every Tx.

Abstract block chain



Merkle proofs are used to prove that a Tx is “on the block chain”

Another application: proof of work

Goal: computational problem that

- takes time $\Omega(D)$ to solve, but
- solution takes time O(1) to verify

(D is called the **difficulty**)

How? $H: X \times Y \to \{0,1,2,\ldots,2^n - 1\}$ e.g. $n = 256$

- puzzle: input $x \in X$, output $y \in Y$ s.t. $H(x,y) < 2^n/D$
- verify(x,y): accept if $H(x,y) < 2^n/D$

Another application: proof of work

Thm: if H is a "random function" then the best algorithm requires D evaluations of H in expectation.

Note: this is a parallel algorithm

$\Rightarrow$ the more machines I have, the faster I solve the puzzle.

Proof of work is used in some consensus protocols (e.g., Bitcoin)

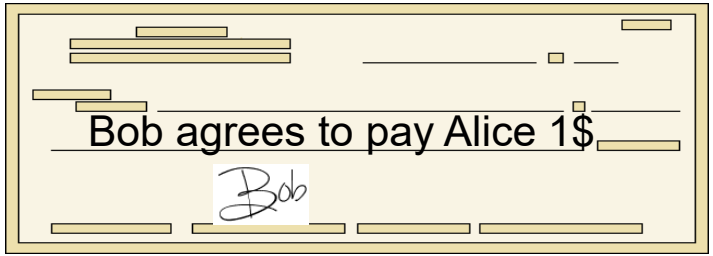
Bitcoin uses $H(x, y) = \mathrm{SHA256}(\mathrm{SHA256}(x.y))$

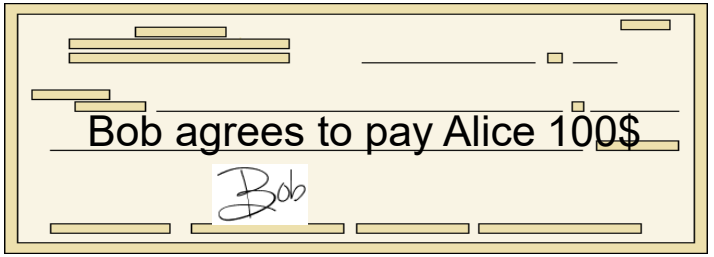
Cryptography background: Digital Signatures

How to authorize a transaction

Signatures

Physical signatures: bind transaction to author



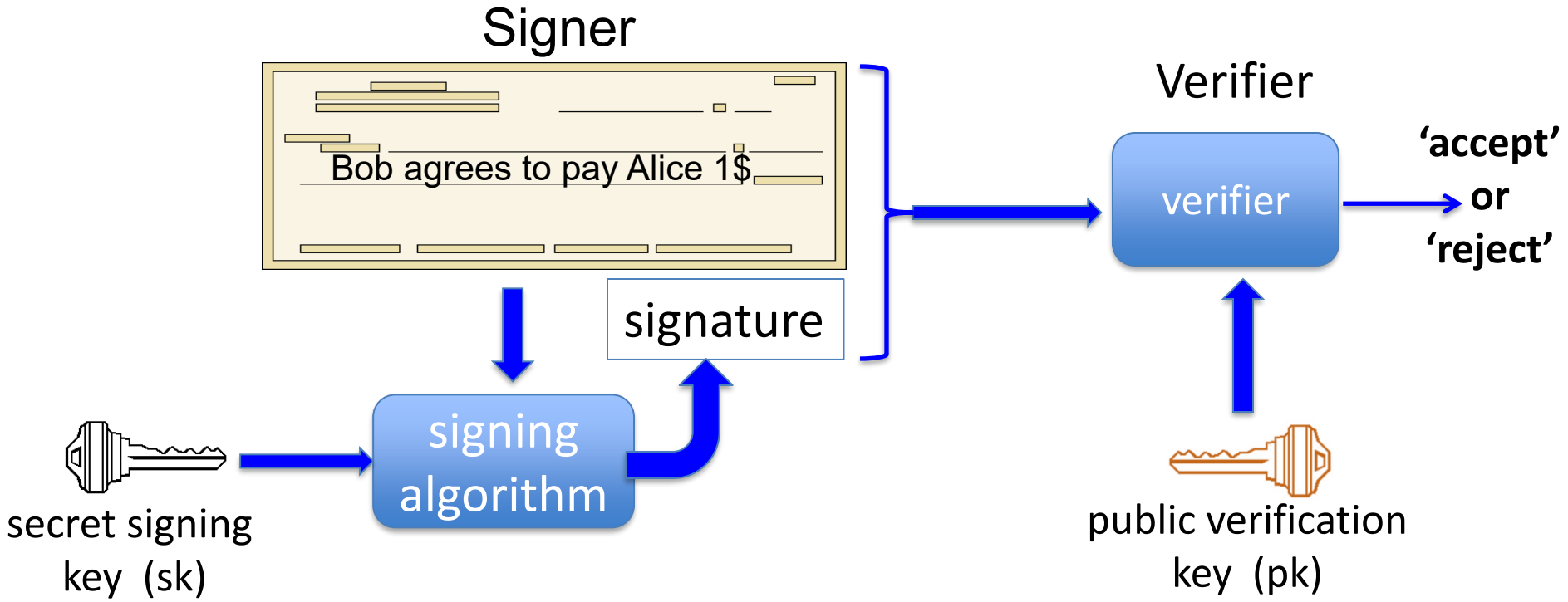


Problem in the digital world:

anyone can copy Bob's signature from one doc to another

Digital signatures

Solution: make signature depend on document



Digital signatures: syntax

Def: a signature scheme is a triple of algorithms:

- **Gen**(): outputs a key pair (pk, sk)
- **Sign**(sk, msg) outputs sig. σ
- **Verify**(pk, msg, σ) outputs 'accept' or 'reject'

Secure signatures: (informal)

Adversary who sees signatures **on many messages** of his choice, cannot forge a signature on a new message.

Families of signature schemes

1. RSA signatures (old … not used in blockchains):
   - long sigs and public keys (≥256 bytes), fast to verify

2. Discrete-log signatures: Schnorr and ECDSA (Bitcoin, Ethereum)
   - short sigs (48 or 64 bytes) and public key (32 bytes)

3. BLS signatures: 48 bytes, aggregatable, easy threshold
   (Ethereum, Solana, Chia)

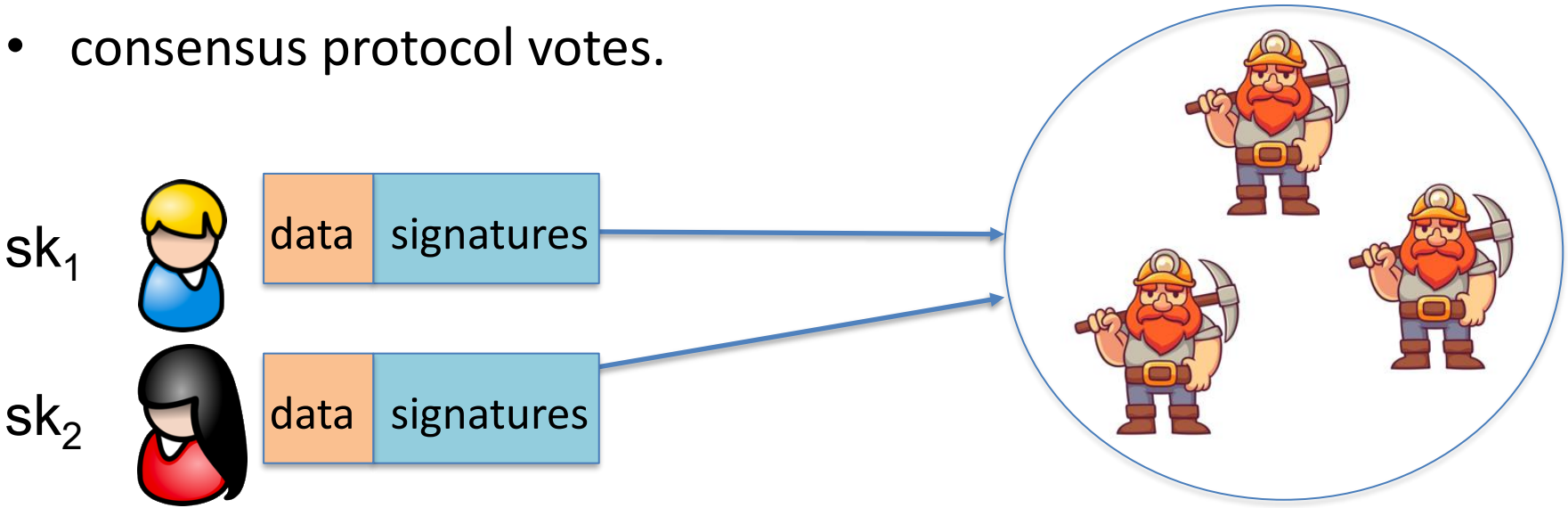
---

4. Post-quantum signatures (ML-DSA): long (≥600 bytes)

Signatures on the blockchain

Signatures are used everywhere:

- ensure Tx authorization,
- governance votes,
- consensus protocol votes.



Readings:

Merkle Trees

(https://decentralizedthoughts.github.io/2020-12-22-what-is-a-merkle-tree/)

END OF LECTURE

Next lecture: the Bitcoin blockchain